

New-Tech Adoption Plan — token_optimizer + session_orchestrator

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Scope:** Adoption analysis + plan for the two nested reusable engines pulled into constitution/ at HEAD e60cbde (§11.4.28(C) depth-1 carve-out). Research only — NO wiring changes made. §11.4.6 no-guessing: every claim below is backed by a command in the “Verified evidence” section; anything not confirmed from the tree is marked UNCONFIRMED:.

0. Where they live (evidenced)

Both engines are **git submodules of the constitution submodule**, hosted under the §11.4.28(C) depth-1 carve-out — NOT consumer submodules of Helix OTA:

- constitution/submodules/token_optimizer/ — module github.com/vasic-digital/token_optimizer (go 1.24)
- constitution/submodules/session_orchestrator/ — module github.com/vasic-digital/session_orchestrator (go 1.22)

constitution/.gitmodules declares both (git@github.com:vasic-digital/token_optimizer.git, git@github.com:vasic-digital/session_orchestrator.git). Each ships a helix-deps.yaml (§11.4.31) and README (.md+.html+.pdf+.docx).

Consumption model = **BY REFERENCE** (§11.4.28(C) / §11.4.80 / §11.4.106 pattern): inherited via the constitution submodule, NEVER copied into Helix OTA, NEVER re-added as a nested own-org submodule of the consumer (that is forbidden; only the constitution submodule itself may host these depth-1). A Go consumer references them by their module path with a replace directive pointing at the in-tree path, or (per each engine’s own helix-deps.yaml) resolves declared own-org deps from the parent project root.

1. token_optimizer

What it is (evidenced — README.md + pkg tree + go test)

A project-agnostic Go engine that sits **in front of an LLM request path** and cuts its token / dollar / byte cost. It is the production cut of the ATM-659 token-reduction research (WS1–WS11). Public request entrypoint: `pipeline.New(cfg *config.Config) (*Optimizer, error) → (*Optimizer).Optimize(req Request, live func(name string) bool) (Decision, error)` (`pkg/pipeline/pipeline.go:221,356`). It binds `cache → router → wire → tier → telemetry`.

Implemented packages (all build + test GREEN, verified below): `pkg/config` (the SINGLE decoupling seam — runtime registry of tiers, pricing, thresholds, alternatives, never-downgrade predicate), `pkg/pipeline`, `pkg/router` (tier decision + never-downgrade HARD floor + failover), `pkg/cache` (exact/semantic/artifact + cross-process file-lock singleflight), `pkg/wire` (`min(TOON, compactJSON)` shape-routed encoder), `pkg/transport` (HTTP + brotli), `pkg/tier` (tier adapters incl. a deterministic `log_triage` tier, `pkg/tier/logtrriage.go:63`), `pkg/telemetry` (JSONL decision spine + savings/p95).

Decoupling contract (README §“Decoupling contract”): ships ZERO project constants. Every project-specific datum (tier name, endpoint, price, threshold, never-downgrade predicate) is registered at runtime through a `*config.Config` (`config.New() → RegisterTier / RegisterAlternative / SetThreshold / SetNeverDowngrade`, `pkg/config/config.go:111,123,172,222,239`). One coupling seam only: `pkg/config`.

How Helix OTA adopts it

Key finding (§11.4.6): Helix OTA's product (`server/`, module `github.com/HelixDevelopment/helix_ota/server`, go 1.26) has **NO LLM request path**. A `grep` for `anthropic|openai|llm|chat/completions|claude` across `server/` + `scripts/` returns only (a) bundled JS UI assets under `server/internal/api/manager-dist/assets/`, (b) a test file `handlers_error_paths_test.go`, and (c) governance/commit script comments — NOT a product LLM call. There is no `scripts/llm/` directory. Therefore `token_optimizer` has **no runtime consumer in Helix OTA today**.

Consequences for adoption:

- **Do NOT wire it into product code now.** Wiring an engine with zero call sites would create dead/unwired code (§11.4.124 — “zero importers $\Rightarrow$ dead” is itself a guess, but here the honest fact is there is no LLM surface to front).
- **Adopt-as-available:** register the engine as a known, build-verified dependency so it is one hop away the moment Helix OTA ships an LLM-backed feature (e.g. an AI release-notes/triage assistant, an LLM log-triage tool for device recordings). The natural first consumer is the deterministic `log_triage` tier (`pkg/tier`) applied to the §11.4.128 device-recording corpus — a free/deterministic first tier before any paid model.
- **Concrete wiring recipe (WHEN a consumer exists):** in the consuming Go module’s `go.mod` add `require github.com/vasic-digital/token_optimizer v0.0.0`
 - replace `github.com/vasic-digital/token_optimizer => ../constitution/submodules/token_optimizer` (path relative to the consumer module), then build the `*config.Config` per the README “How a consumer registers its data” example and call `pipeline.New / Optimize`. The engine’s own six own-org deps (`helix-deps.yaml`: TOON, Embeddings, VectorDB, Normalize, LLMProvider, conversation) are only pulled by the packages that use them — the current request path builds with just `github.com/andybalholm/brotli` (`go.sum`), so a minimal consumer needs NO extra own-org submodules.

Separate, immediately-applicable win — §11.4.141 token-efficiency: the §11.4.141 mandate (cut agent token spend to 30–40%) is about the **Claude Code development workflow** (prompt-cache the governance prefix, tier mechanical subagents to Haiku, thin literal-anchor index, CodeGraph-first). That is Claude-Code config + the existing `CLAUDE_ANCHORS_FULLL.md` index pattern this repo already uses — it is NOT the `token_optimizer` Go engine and does not require wiring `token_optimizer`. `token_optimizer` is the runtime-LLM-request optimizer; §11.4.141 is the agent-context optimizer. Keep them distinct.

Risk / blast-radius

Near-zero if not wired (research + build-verify only). If wired later: contained to the single `pkg/config` seam; the never-downgrade HARD floor prevents a load-bearing request being routed to a cheaper tier. No product code is touched by adoption-now.

Classification: (b) needs review → effectively adopt-as-available

Not (a) safe-to-wire-now, because there is no consumer — wiring now would be dead code. Not (c) operator-blocked. The safe non-destructive step now is: build-verify + document + track a workable item to wire it at the first LLM surface. Recommend a Helix OTA Task item: “Register token_optimizer as LLM-request front for future AI features; first candidate = deterministic log_triage over device-recording corpus (§11.4.128).”

2. session_orchestrator

What it is (evidenced — README.md + package tree + go test)

A project-agnostic Go engine coordinating a **flowing pool of session aliases behind a floating orchestrator role**, with a fail-closed health predicate deciding which alias may work right now. This is a **multi-agent dev-orchestration** engine (the machinery behind §11.4.103 continuous parallel-stream + §11.4.176 exactly-once claim + §11.4.119 single-owner), NOT product code.

Implemented, build+test GREEN packages: - alias/ — concurrency-safe registry (aliases by NAME only, never credentials) + Classify (maps a probe result onto the health taxonomy, catches an HTTP-200 body that hides a plan-cap string) + IsOperable (total fail-closed predicate, no fail-open path) + SortByPriority/FirstOperable. - claim/ — the flowing-pool claim registry: exactly-once, deadlock-free, **single-owner** binding of an alias to at most one work-unit; TryClaim non-blocking atomic CAS (GRANTED/GRANTED_EXISTING/DENIED), Release/Renew + TTL/dead-holder reaping; append-only Event log (claim/claim.go). - scheduler/ — Schedule(reg, cr, workUnits, cfg) (Result, error) (scheduler/scheduler.go:133): the **non-failover** placement pass, pure composition of alias.IsOperable + claim.TryClaim; a work-unit with no claimable operable alias is returned **explicitly Unassigned** (never dropped, never double-assigned); injected clock ⇒ deterministic. - supervisor/ — death-detection watchdog: classifies entities ALIVE/SUSPECT/DEAD from heartbeat age + an injectable liveness proof; a **signal, not an action** (emits verdict + transition log, performs no recovery).

Honest boundary (README §“Not yet implemented”): the same-session failover/resume spine (capture session id → detect limit → quiesce → atomic swap → `claude --resume` on another alias) is **NOT implemented** — its `CROSS-CLAUDE_CONFIG_DIR claude --resume` continuity premise is UNCONFIRMED: pending a POC. So the engine today gives you assignment/health/claim/death-signal primitives, but NOT automatic re-homing of a degraded stream.

Decoupling contract: hardcodes no track/alias/directory/threshold; an Alias holds NO secret material (caller runs the probe, passes only the observable result). `helix-deps.yaml`: `deps: []` — pure Go stdlib, zero external dependencies.

How Helix OTA adopts it

Helix OTA already runs a heavy parallel-development methodology (§11.4.58/§11.4.103) and needs, per constitution, an exactly-once work-claim registry (§11.4.176) and single-owner device-lock (§11.4.119) for parallel device testing. `session_orchestrator`'s `claim + scheduler` packages are a ready-made, test-covered, deterministic implementation of exactly those primitives.

- **Adoption path:** consume BY REFERENCE from a Helix OTA dev-tooling Go module (e.g. a small `tools/` or `scripts/` Go command that manages the parallel-stream / device-claim registry), via `require github.com/vasic-digital/session_orchestrator + replace => ../constitution/submodules/session_orchestrator`. The consumer registers its own tracks/devices at runtime (alias = a track or a device handle), runs its own probe, and calls `Schedule` for placement + `claim.TryClaim/Release` for the exactly-once device lock. Zero own-org deps to pull.
- **What it does NOT give you yet:** automatic failover/re-homing of a crashed or rate-limited stream (the UNCONFIRMED: spine). Helix OTA's §11.4.147 crashed-agent respawn-until-complete would still be conductor-driven on top of the death-detection signal, not automatic inside the engine.

Risk / blast-radius

Zero to product runtime (dev-tooling only). If adopted for the device-claim registry, blast-radius is the parallel-testing coordination layer — contained, and the fail-closed predicate + single-owner CAS are designed to be safe under contention. No secret material flows through it.

Classification: (b) needs review

Safe-to-build/test now; safe to prototype a dev-tooling consumer for the §11.4.176/§11.4.119 device-claim registry. Full adoption needs a design review because (i) the failover/resume spine is UNCONFIRMED:/absent, so Helix OTA must NOT assume automatic re-homing, and (ii) it would replace/compose with any existing ad-hoc claim/lock mechanism the project already has, which must be reconciled (§11.4.120), not duplicated. Recommend a Helix OTA Task item: “Prototype session_orchestrator claim+scheduler as the §11.4.176 exactly-once / §11.4.119 single-owner device-claim registry for parallel device testing; conductor-side failover remains until the engine’s WS-C spine lands.”

3. Summary classification

Engine	Product runtime consumer today?	Build/test	Safe-to-wire-now	Verdict
token_optimizer	NO (server has no LLM path)	GREEN (9 pkgs)	No consumer ⇒ wiring = dead code	(b) adopt-as-available — register + track; wire at first LLM surface
session_orchestrator	NO (dev-orchestration tool)	GREEN (4 pkgs)	Prototype-yes for device-claim registry	(b) needs review — prototype claim+scheduler for §11.4.176/§11.4.119; no auto-failover yet

Neither engine is (a) safe-to-wire-into-product-now (no product consumer for either) and neither is (c) operator-blocked. Both are build-verified, decoupled, consumed BY REFERENCE, and ready to adopt behind a small dev/tooling consumer or the first LLM feature. The immediately-actionable \$11.4.141 token-efficiency wins for THIS project's Claude workflow are independent of the token_optimizer Go engine (governance prompt-caching + subagent Haiku-tiering + literal-anchor index — Claude Code config, not Go wiring).

4. Verified evidence (commands run + key output)

1. `find constitution -maxdepth 4 -type d \( -iname '*token*' -o -iname '*session*' -o -iname '*orchestrat*' \) → constitution/submodules/session_orchestrator, constitution/submodules/token_optimizer (+ constitution/tests/token_efficiency, constitution/docs/research/token_efficiency).`
2. `git -C constitution rev-parse HEAD → e60cbde2e33e7d0542a937331763f6be7e739c9a. cat constitution/.gitmodules → both engines declared with vasic-digital SSH URLs.`
3. Read `constitution/submodules/token_optimizer/README.md` (Revision 2) + `constitution/submodules/session_orchestrator/README.md` (Revision 2, “early scaffold”).
4. `cat ../token_optimizer/helix-deps.yaml → 6 own-org deps (TOON, Embeddings, VectorDB, Normalize, LLMProvider, conversation), all layout: flat. cat ../session_orchestrator/helix-deps.yaml → deps: [] (pure stdlib). token_optimizer/go.mod → module github.com/vasic-digital/token_optimizer, go 1.24, only github.com/andybalholm/brotli v1.2.2. session_orchestrator/go.mod → github.com/vasic-digital/session_orchestrator, go 1.22.`
5. `cd token_optimizer && go build ./... && go test ./... → BUILD_EXIT=0; all 9 packages ok (cache, config, pipeline, router, telemetry, tier, transport, transport/brotli, wire).`
6. `cd session_orchestrator && go build ./... && go test ./... → BUILD_EXIT=0; all 4 packages ok (alias, claim, scheduler, supervisor).`
7. `grep -nE '^func |^type ' pkg/config/config.go → config.New(), RegisterTier, RegisterAlternative, SetThreshold, SetNeverDowngrade, IsForbiddenDowngrade (the single decoupling seam). pkg/pipeline/pipeline.go → New(cfg *config.Config), Optimize(req Request, live func(name string) bool). pkg/tier/`

logtrriage.go → TierLogTriage = "log_trriage" deterministic tier.
scheduler/scheduler.go → Schedule(reg, cr, workUnits, cfg);
claim/claim.go → TryClaim/Outcome/Event/Registry.

8. Helix OTA server LLM-path check: head -5 server/go.mod →
module github.com/HelixDevelopment/helix_ota/server, go 1.26.
grep -rile 'anthropic|openai|llm|chat/completions|claude'
server scripts → only bundled JS assets (manager-dist/assets/
*.js), a test file, and governance/commit-script comments — NO
product LLM call. scripts/llm/ does not exist. ⇒ token_optimizer
has no runtime consumer today.
9. Read §11.4.141 full text in constitution/
CLAUDE_ANCHORS_FULL.md:649 — confirms token-efficiency mandate
targets the Claude Code agent workflow (governance prompt-
caching, subagent tiering, literal-anchor index), distinct from the
token_optimizer runtime engine.